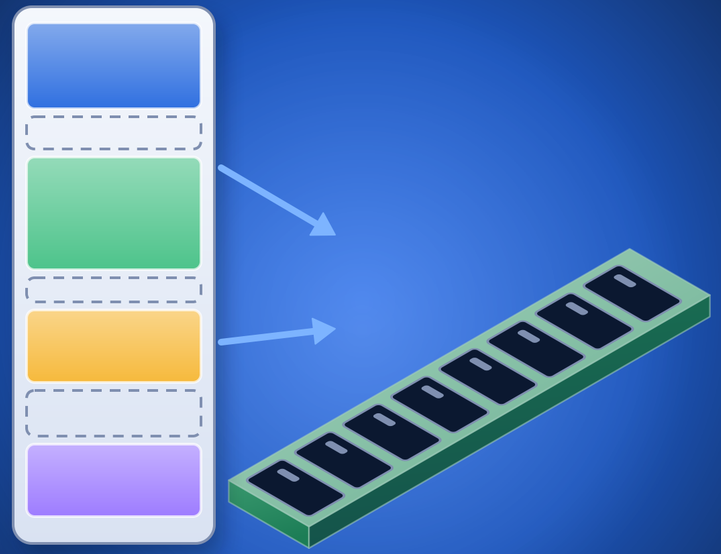
Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · WEEK 12

Memory Management



How programs are placed in main memory: address binding, contiguous allocation, fragmentation, paging and swapping.

Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Explain the difference between a logical and a physical address, and the role of the MMU.
2. Apply first-fit, best-fit and worst-fit to allocate memory contiguously.
3. Distinguish internal from external fragmentation.
4. Translate logical addresses to physical addresses in a paging system with a TLB.
5. Describe hierarchical, hashed and inverted page tables.
6. Describe swapping, and address translation on IA-32, x86-64 and ARM.

THIS WEEK

Topic 10

A program must be in main memory to run, and many programs want to be there at once.

DISCUSSION

Two programs both store a value at address 1000

They run at the same time, and neither overwrites the other. How can both be using address 1000?

PART 1

Background

What the CPU sees, and how each process is kept inside its own memory.

Week 12

Memory and the CPU

A program must be in memory

It is brought from disk into memory and placed within a process before it can run.

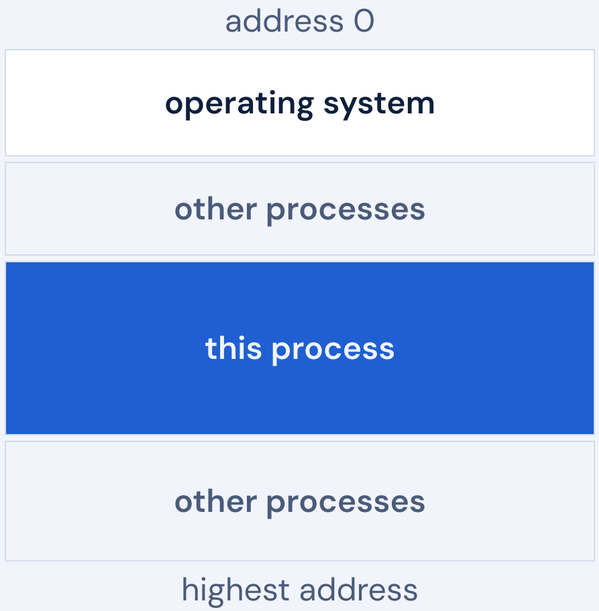
What the CPU can reach

Only main memory and the registers can be accessed directly. A register takes one CPU clock. Main memory can take many cycles, causing a stall.

What memory sees

A stream of addresses with read or write requests. It knows nothing about which program produced them, or why.

Protection with base and limit registers



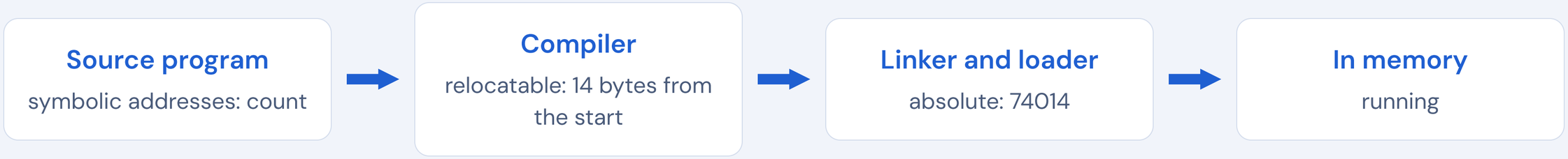
A pair of registers defines the legal addresses of the running process.

Base is the smallest legal address. Limit is the size of the range. With base 400000 and limit 150000, the process may use 400000 to 549999.

The CPU checks every address generated in user mode: base ≤ address < base + limit. Anything else traps to the operating system.

Only the operating system, in kernel mode, can load the two registers.

Address binding



Compile time

If the location is known in advance, absolute code is generated. Move the program and it must be recompiled.

Load time

The compiler generates relocatable code, and the final binding is done when the program is loaded.

Execution time

Binding is delayed until run time, so the process can be moved while it runs. This needs hardware support, and it is what most systems do.

Logical and physical addresses

WHAT THE PROGRAM SEES

Logical address

Generated by the CPU. Also called a virtual address. The set of all of them is the logical address space of the process.

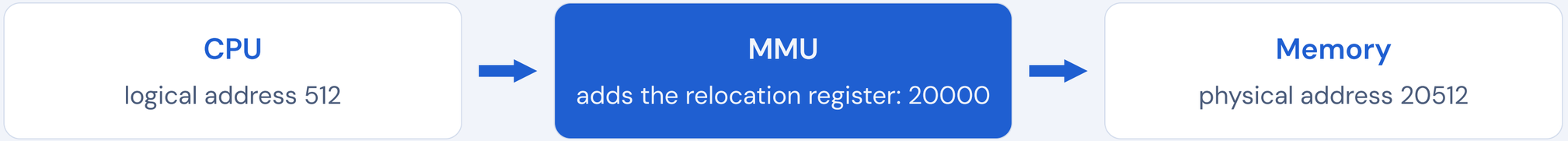
WHERE THE DATA REALLY IS

Physical address

The address seen by the memory unit: the one loaded into the memory-address register.

With compile-time and load-time binding the two are the same. With execution-time binding they differ, and something must translate one into the other.

The memory-management unit



The MMU is the hardware that maps logical to physical addresses at run time. The simplest scheme adds the value of a relocation register to every address. The program works only with logical addresses and never sees a physical one.

Two programs, one address

PROCESS A

Program A

Logical address 1000. Its relocation register holds 20000. The MMU sends the access to physical address 21000.

PROCESS B

Program B

Logical address 1000. Its relocation register holds 60000. The MMU sends the access to physical address 61000.

This answers the opening question. Each process has its own logical address space starting at 0. The operating system loads the registers for whichever process is running.

Dynamic loading and dynamic linking



Dynamic loading

A routine is not loaded until it is called. Routines that are never used, such as rare error handlers, never take up memory. It needs no special support from the operating system.



Dynamic linking

Linking is postponed until execution time. A dynamically linked library is loaded when first needed, and one copy is shared by every process that uses it. A library can be updated without relinking the programs.

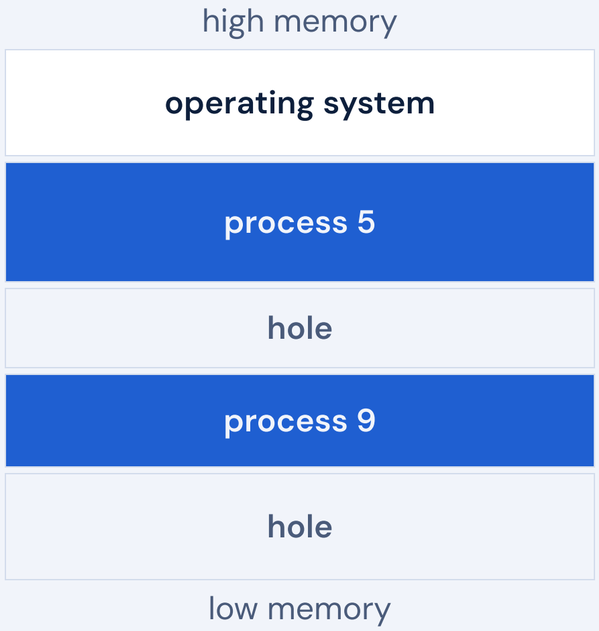
Static linking, by contrast, copies the library code into every executable file.

PART 2

Contiguous memory allocation

Each process in one single, continuous block of memory.

Contiguous allocation



Memory has two partitions: one for the operating system, one for user processes.

Each process occupies one contiguous section. The relocation register and the limit register protect processes from each other.

Variable partitions: a block of exactly the needed size is given to each process. A hole is a block of free memory. When a process ends, its block becomes a hole, and adjacent holes are merged.

Which hole to use?

Free holes, in KB	300	120	500	200

Strategy	Rule	A request of 180 KB goes to	Left over
First fit	The first hole that is big enough	the 300 KB hole	120 KB
Best fit	The smallest hole that is big enough	the 200 KB hole	20 KB
Worst fit	The largest hole	the 500 KB hole	320 KB

Best fit and worst fit must search the whole list, unless it is kept sorted by size. First fit and best fit are better than worst fit in speed and in use of memory; first fit is generally the fastest.

Fragmentation

BETWEEN THE BLOCKS

External fragmentation

Enough total free memory exists to satisfy a request, but it is not contiguous. It is split into many small holes.

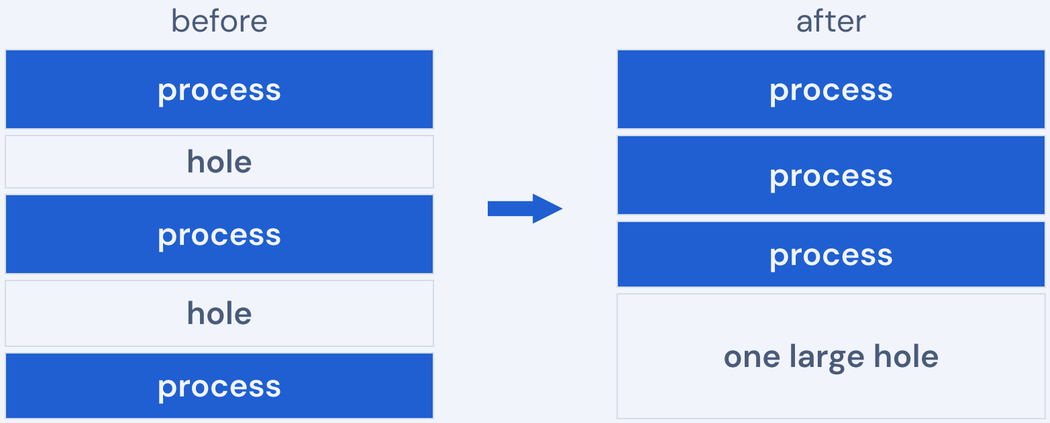
INSIDE A BLOCK

Internal fragmentation

The memory given to a process is slightly larger than what it asked for. The difference is inside its block and is not used.

The 50-percent rule: with first fit, for every N allocated blocks about 0.5 N more are lost to fragmentation, so a third of memory may be unusable.

Compaction



Shuffle the contents of memory to bring all the free memory together in one block.

It is possible only if relocation is dynamic and done at execution time.

It is expensive. The other answer to external fragmentation is to stop requiring that a process be contiguous. That is paging.

PART 3

Paging

The physical address space of a process no longer has to be contiguous.

The idea of paging

Frames

Physical memory is divided into fixed-size blocks called frames. The size is a power of 2, from 4 KB up to large pages of megabytes or gigabytes.

Pages

Logical memory is divided into blocks of the same size, called pages. A process of N pages needs N free frames, anywhere.

Page table

One per process. It records, for each page, the frame that holds it.

No external fragmentation: any free frame can be used. Some internal fragmentation remains, in the last page of each process.

Translating an address



The page number is an index into the page table. The offset is copied unchanged. With a logical address space of 2^m bytes and a page size of 2^n bytes, the high m - n bits are the page number.

A small example

Page	Frame
0	6
1	2
2	7
3	1

Page size 4 bytes. Logical memory 16 bytes: 4 pages. Physical memory 32 bytes: 8 frames.

Logical address 5. 5 = 1 × 4 + 1: page 1, offset 1. Page 1 is in frame 2. Physical address = 2 × 4 + 1 = 9.

Logical address 14. 14 = 3 × 4 + 2: page 3, offset 2. Page 3 is in frame 1. Physical address = 1 × 4 + 2 = 6.

Divide by the page size: the quotient is the page number and the remainder is the offset.

Internal fragmentation in paging

70,000

Bytes in the process

With a page size of 4,096 bytes.

18

Frames allocated

17 full pages hold 69,632 bytes, and 368 bytes go in an 18th.

3,728

Bytes wasted

4,096 - 368, in the last frame.

On average half a frame per process is wasted. Small pages waste less, but make the page table larger. Page sizes have grown over time.

Implementing the page table



Kept in main memory

The page-table base register, PTBR, points to the table of the running process. A context switch changes only this register.



The cost

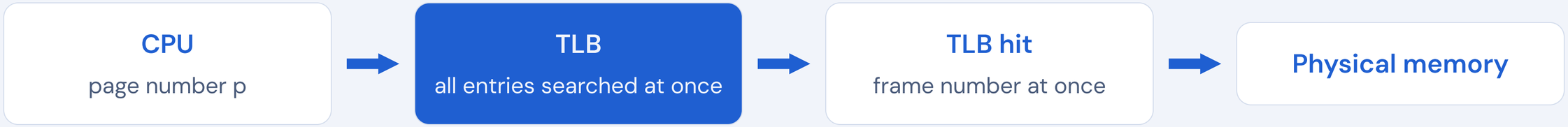
Every access to memory now needs two: one for the page-table entry and one for the data. Memory would be twice as slow.



The cure

A small, fast hardware cache of recent translations: the translation look-aside buffer.

The translation look-aside buffer



TLB miss

The page number is not in the TLB. The page table in memory is consulted, and the translation is added to the TLB.

Small

Typically 64 to 1,024 entries. When it is full, an entry is replaced. Some entries can be wired down so they are never removed.

Address-space identifiers

Some TLBs store an ASID in each entry to identify the process, so the TLB need not be emptied at every context switch.

Effective access time

$$EAT = h \times t + (1 - h) \times 2t$$

h is the hit ratio: the fraction of translations found in the TLB. t is the time of one memory access.

A hit needs one memory access. A miss needs two: the page table, then the data.

Hit ratio	Calculation, t = 100 ns	EAT
85%	0.85 × 100 + 0.15 × 200	115 ns
99%	0.99 × 100 + 0.01 × 200	101 ns

With a 99% hit ratio, paging slows memory access by only 1%.

Protection and shared pages



Protection bits

Each page-table entry carries bits that say whether the page may be read, written or executed. An illegal access traps to the operating system.



Valid–invalid bit

Valid: the page is in the process's logical address space. Invalid: it is not. An access to an invalid page traps.



Shared pages

One copy of read-only code, such as a standard library, is mapped into the page tables of many processes. Each process keeps its own data pages.

PART 4

The structure of the page table

A flat page table for a modern address space would be enormous.

The problem of size

2^{32}

Bytes of logical address space

A 32-bit address.

2^{20}

Pages of 4 KB

About one million page-table entries.

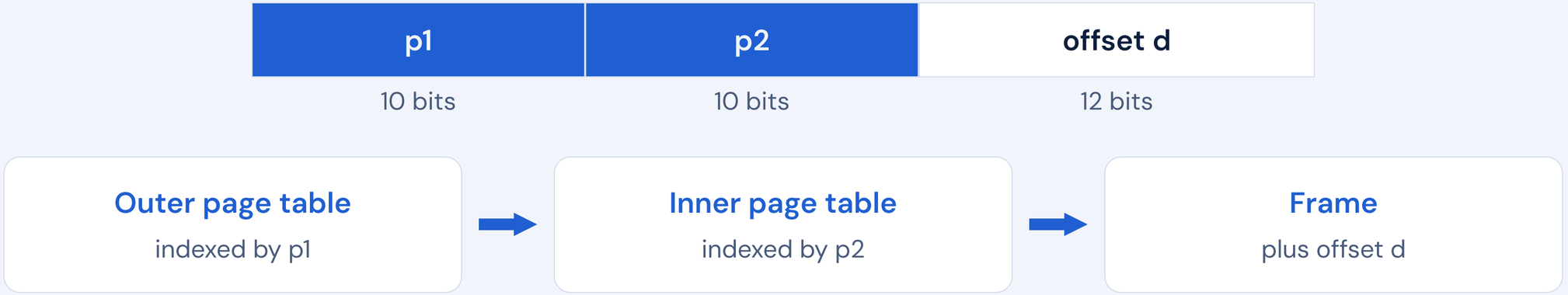
4 MB

For one page table

At 4 bytes per entry, for each process, and contiguous in memory.

Three solutions: hierarchical paging, hashed page tables and inverted page tables.

Hierarchical paging



Page the page table. The 20-bit page number is split in two. Only the inner tables that are actually used need to exist. A 64-bit address space would need too many levels for this to work.

Hashed and inverted page tables

FOR LARGE ADDRESS SPACES

Hashed page table

The virtual page number is hashed into a table. Each entry holds a chain of elements that hash to the same place: the virtual page number, the frame, and a pointer to the next. The chain is searched for a match.

ONE TABLE FOR ALL

Inverted page table

One entry for each real frame of memory, not for each page. The entry says which process and which virtual page is in that frame. One table for the whole system: less memory, but a longer search, so a hash table is used to speed it up.

Segmentation



A program is a collection of segments: logical units of different sizes.

A logical address is a pair: segment number and offset. A segment table gives each segment's base and limit. An offset beyond the limit traps.

Segments match how a programmer thinks, and each can have its own protection. But they vary in size, so external fragmentation returns.

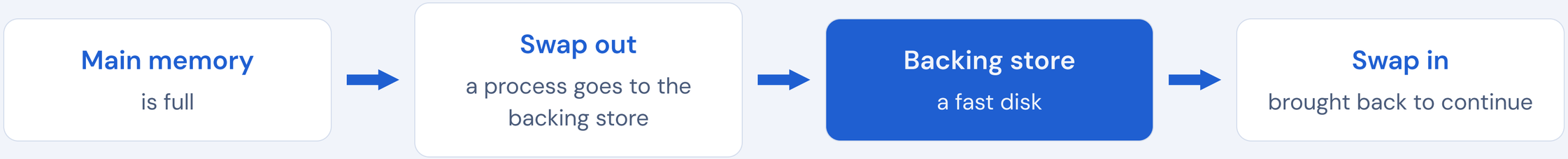
Some architectures combine the two and page each segment.

PART 5

Swapping

When the processes together need more memory than there is.

Swapping



Standard swapping

Whole processes are moved. Most of the swap time is transfer time, which grows with the amount of memory moved.

Swapping with paging

Only some pages of a process are moved: page out and page in. This is what Linux and Windows do.

On mobile systems

Flash storage is small and wears out with writes, so there is usually no swapping. iOS asks applications to give up memory. Android ends a process after saving its state.

Why standard swapping is rarely used

200 MB

Process size

To be swapped out.

50 MB/s

Transfer rate

Of the backing store.

4 s

To swap out

And 4 seconds more to swap another process of the same size in.

Eight seconds for one context switch. A process doing I/O is a further problem: it cannot be swapped out while I/O is writing into its memory.

PART 6

Real architectures

Intel and ARM.

Week 12

Intel IA-32 and x86-64

32 BITS

IA-32: segmentation, then paging

The CPU generates a logical address: a segment selector and an offset. The segmentation unit turns it into a linear address, and the paging unit turns that into a physical address. Pages are 4 KB or 4 MB, with two-level paging.

64 BITS

x86-64

A 64-bit address would allow more memory than any machine holds. In practice 48 bits are used, with four levels of paging and pages of 4 KB, 2 MB or 1 GB.

Page address extension let 32-bit processors reach more than 4 GB of physical memory, by adding a third level of paging and widening the entries to 64 bits.

ARM

Where it is used

The dominant architecture for mobile devices, and now in laptops and servers too.

ARMv8

A 64-bit architecture with up to four levels of paging. Pages of 4 KB, 16 KB and 64 KB, and larger contiguous regions.

Two levels of TLB

Small, fast micro-TLBs, one for data and one for instructions, backed by a larger main TLB.

Key takeaways

01

Two kinds of address

The CPU generates logical addresses. The MMU turns them into physical ones at run time.

02

Contiguous allocation fragments

First fit, best fit and worst fit all leave holes: external fragmentation.

03

Paging

Fixed-size pages map onto any free frames through a page table, and the TLB makes it fast.

04

Large address spaces

Hierarchical, hashed and inverted page tables keep the tables manageable.

Next session

COMING UP

Virtual memory

Demand paging, page faults, page-replacement algorithms, thrashing and the working-set model.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 9.